OPENER: Open Partitioning Embedding for Named Entity Recognition

Anonymous Author(s)
Paper submitted for double-blind review
LyRIDS Symposium 2026

Abstract

Named Entity Recognition (NER) is a foundational task in Natural Language Processing, used to extract entities from raw text. However, real-world NER applications rarely afford the luxury of a static, predefined set of entity classes. This limitation has driven the emergence of open-world NER, where systems must assign entity types that were not fixed at training time. To tackle this challenge, recent systems either train a dedicated encoder or prompt billion-parameter language models, and they are compared almost exclusively on accuracy, ignoring their compute and energy cost. We present OPENER, an open-world NER pipeline that redefines the optimal trade-off between accuracy, latency, and energy frugality. OPENER is assembled from ready-to-use components: a frozen mention detector, a Matryoshka-truncatable sentence embedder sharpened by a light contrastive fine-tuning with error-driven hard-negative mining, and a swappable typing head, so that no encoder is trained from scratch. This design relies on transfer learning: it reuses pretrained components instead of training a dedicated encoder or prompting a large language model, a choice still uncommon in open-world NER. The typing head comes in two forms. OPENER-ZS is *zero-shot*: it types each mention against label-name prototypes and needs no target labels, for the truly open-world setting where entity types are not known in advance. OPENER-Sup instead fits a lightweight supervised probe when a domain's entity classes are fixed and labelled, trading openness for accuracy. We benchmark OPENER against GLiNER, GNER, OWNER and a 4-bit Qwen2.5-1.5B large language model on 13 datasets along three axes at once: quality (measured by Adjusted Mutual Information, AMI), inference latency (median per-sentence p50), and energy consumption (watt-hours and grams of CO_2eq per run). Throughout, we separate entity typing on gold mentions (given perfect mention boundaries) from the full end-to-end setting to analyze each component. Moreover, our energy-aware comparison exposes order-of-magnitude efficiency gaps that a quality-only evaluation would hide. On our 13-dataset benchmark, both variants achieve the highest scores in their category, averaged over three embedder-retraining seeds. OPENER-ZS is the most accurate *zero-shot* system end-to-end (39.5 AMI, ahead of GLiNER-L's 38.9 and OWNER's 34.3) while staying frugal, at 1.7 Wh and $\sim$180 ms per sentence: several times less energy and latency than OWNER (14.3 Wh, 616 ms), and far below the 4-bit Qwen LLM (11.4 Wh, 5 s). Given a domain's labels, OPENER-Sup is the most accurate system overall (40.1 AMI). On gold mentions, with detection removed, typing alone reaches 62.3 AMI versus OWNER's 43.0: the wide gap to the end-to-end scores pins the open-world bottleneck on mention detection rather than typing, the clearest axis for future gains. We release the code, the per-experiment configurations, the fine-tuned embedders, and the full benchmark.

Index Terms

named entity recognition, open information extraction, open-world NER, supervised named entity recognition, unsupervised named entity recognition, zero-shot NER, energy efficiency, frugal AI

I. Introduction

Named Entity Recognition (NER) is a critical and challenging task in the field of Natural Language Processing (NLP). Its primary goal is to locate and classify key elements in unstructured text into predefined categories (e.g., extracting "Marie Curie" as a PERSON or "1898" as a DATE). Its most widespread uses include automating the analysis of legal or medical documents, intelligently sorting customer support tickets, or even searching and finding this research paper itself. However, real-world NER rarely operates with a static label set, as novel categories inevitably emerge across new domains. Although traditional models accurately assign contiguous sequences of tokens, called spans, into fixed classes, this rigid training renders them blind to unobserved types, such as identifying "radium" as a CHEMICAL [1]. To avoid the slow and costly process of re-annotating data and retraining from scratch, *open-world* NER is designed to dynamically recognize and type entities without relying on a frozen training inventory.

Notable examples of open-world NER models include: GLiNER [2] scores candidate spans against the target type names supplied as queries in a single forward pass, while GNER [3] recasts NER as text generation with a sequence-to-sequence model. Both can type entities whose categories were not fixed at training time, at the cost of training a dedicated model for the task. A complementary line prompts or instruction-tunes billion-parameter language models, such as UniversalNER [4] and GoLLIE [5], which are accurate but slow, brittle, and often too large for commodity hardware. Even a small quantized instruction-tuned model that does fit, such as Qwen2.5-1.5B-Instruct [6] in 4-bit, which we evaluate, stays comparatively slow and costly. OWNER [7] takes yet another route, training a dedicated entity encoder with a triplet objective and clustering the resulting mentions into dynamically named types. But it still trains an encoder from scratch and pays a heavy per-sentence inference cost.

Across all of these, evaluation is reported almost exclusively on accuracy: the compute, latency, and energy consumption that separate a 467 M-parameter pipeline from a 7 B-parameter model are left out of the comparison, even though they decide what can actually be deployed.

We present **OPENER** (**O**pen **P**artitioning **E**mbedding for **N**amed **E**ntity **R**ecognition), an open-world NER pipeline built for all three axes at once: on our 13-dataset benchmark, it is the most accurate of the compared systems while remaining fast and energy-frugal. Unusually, we train no encoder from scratch. It is assembled entirely from ready-to-use and pretrained components, a design still uncommon in open-world NER. Concretely, a frozen mention detector proposes spans, a pretrained Matryoshka sentence embedder (sharpened by a light contrastive fine-tuning and an error-driven hard-negative mining pass) maps each span to a vector, and a typing head assigns a type. The pipeline is modular: mention detector, embedder, and typing head are independently swappable. We vary each in the ablations, testing many combinations to reach the best pipeline: closed-set detectors (BERT [8], RoBERTa [9]) against GLiNER [2] at three sizes, several embedders (BGE [10], MPNet [11], mxbai [12], E5-base [13] and Nomic [14]), the best Matryoshka [15] dimension to trade quality for computation, and different algorithms for entity typing.

The typing head exposes two operating points: OPENER-ZS, the label-free zero-shot head we put forward as the open-world default, and OPENER-Sup, a supervised upper bound for domains whose entity classes are fixed and labelled. We detail both below.

Our contributions are as follows.

- **The OPENER pipeline, a new modular architecture**: to the best of our knowledge, it is the first open-world NER pipeline assembled entirely from ready-to-use, pretrained components, with no encoder trained from scratch. Instead, it relies on *transfer learning* through a frozen mention detector and a Matryoshka embedder, where only the latter undergoes light fine-tuning. The pipeline's three stages (mention detection, embedding, and typing) are *independently swappable*, and the typing head itself supports two interchangeable modes (zero-shot or supervised). We consider this plug-and-play modularity to be a novel contribution to open-world NER. The embedder is lightly refined in two steps: an initial contrastive fine-tuning to cluster same-type mentions, followed by an *error-driven hard-negative mining* pass targeting the most frequently confused type pairs. This second step improves typing-on-gold AMI by 8.5 points compared to standard contrastive fine-tuning, successfully demonstrating that a general-purpose pretrained representation transfers well enough to type open-world entities.
- **OPENER-ZS**: it is a label-free typing head that does not need target labels and combines three signals:
  - *Label-name prototypes*: we embed each candidate type *name* (the words "person", "location", ...) into a prototype vector and assign a mention to its nearest prototype: so "Marie Curie" is typed PERSON because her mention vector lands closest to the embedding of the word "person".
  - A *transductive refinement*: the prototypes are re-estimated from the test mentions themselves (not just the label names), tightening each type to the domain at hand without using any labels.
  - As *a fusion* with the detector's own zero-shot type guess, GLiNER already proposes a type when it detects "Marie Curie", a signal that prototype matching alone would discard. It is the best zero-shot system end-to-end on our benchmark, ahead of GLiNER-L, GNER and OWNER under an identical protocol.
- **OPENER-Sup**: a supervised operating point for domains whose entity classes are fixed and labelled. Where OPENER-ZS matches each mention to a label *name*, OPENER-Sup instead *learns* the geometry of each type from labelled examples. The principle is the following: once mentions are embedded, a linear classifier carves the high-dimensional embedding space into regions, one per entity type, separated by hyperplanes. A new span is then typed by which region its vector falls into: a mention landing in the "chemical" region is typed CHEMICAL, because same-type mentions cluster together in that space. We use a *balanced* linear support-vector classifier (LinearSVC), where "balanced" re-weights each class inversely to its frequency so that rare types are not drowned out by frequent ones, a common issue since entity types are heavily imbalanced. This trades openness for accuracy: it is the most accurate configuration in our benchmark, the best end-to-end AMI of all compared systems, and the best typing on gold mentions, but it needs target labels.
- **An open-world NER latency and energy-aware benchmark** on 13 datasets that reports quality, latency, and energy jointly, separates typing-on-gold from end-to-end, and shows that the dominant open-world bottleneck is mention detection rather than entity typing. This is why an *open-world* detector is essential: closed-set NER detectors (BERT/RoBERTa) collapse on specialized domains, an efficiency/quality gap an accuracy-only evaluation would hide. The comparison also includes a small instruction-tuned LLM, Qwen2.5-1.5B-Instruct in 4-bit (the largest generative model that fits our commodity GPU), which turns out to be both the least accurate and, per sentence, the most expensive of all systems (on a 150-sentence subsample, Table V). All OPENER results are averaged over three full embedder-retraining seeds (7, 42, 123) and reported as mean $\pm$ std. The compared baselines are frozen checkpoints whose inference is deterministic. The differences we report are therefore robust to initialisation.

The rest of the paper is organised as follows. section II reviews related work, section III details the OPENER pipeline, section IV presents the benchmark and ablations, and section V concludes.

II. Related Work

OPENER sits at the intersection of three lines of research: open-world and zero-shot NER, pretrained entity representations refined by contrastive learning, and energy-aware machine learning. We review each in turn and state how OPENER differs.

A. Closed-set and Span-based NER

Traditional NER is cast as supervised sequence labeling over a fixed inventory of entity types, typically with a pretrained encoder followed by a token or span-level classifier. Early neural taggers replaced hand-made features with recurrent encoders over a CRF layer: the BiLSTM-CRF [16] paired character and word-level representations, and the BiLSTM-CNN-CRF [17] added a character-level CNN to obtain a fully end-to-end sequence labeler. The pretrained encoder paradigm subsequently increased in-domain accuracy further, fine-tuning a bidirectional transformer such as BERT [8] with a lightweight classification head. Beyond flat tagging, span-based formulations enumerate candidate spans and classify them directly, which simplifies the handling of nested and discontinuous entities [1]: biaffine NER [18] scores every span with a biaffine layer borrowed from dependency parsing, while an MRC framework [19] recasts typing as machine reading comprehension, querying the sentence with a natural-language description of each type, a use of label semantics our zero-shot head also exploits. These models reach high accuracy on in-domain benchmarks such as CoNLL-2003 [20], as documented in a comprehensive survey of deep learning for NER [21], but their label set is frozen at training time. Adding a new entity type requires re-annotating data and retraining the model, which is the central limitation that open-world NER seeks to remove.

B. Open-world and Zero-shot NER

Open-world NER aims to recognise entity types that were not fixed at training time. GLiNER [2] treats the candidate type names as queries to a bidirectional encoder, a DeBERTaV3 backbone [22], and scores every span against them in a single forward pass, *via a span-type similarity matrix*, offering a fast alternative to autoregressive tagging. GNER [3] instead casts NER as text generation with a sequence-to-sequence model [23]: it regenerates the input sentence with inline type tags (e.g. "`Marie Curie(person) discovered radium(chemical)`"), and shows that explicitly modeling the negative, non-entity tokens in this output sharpens type boundaries. Close to our work, OWNER [7] is an unsupervised open-world pipeline that decouples mention detection from entity typing: it embeds detected mentions with an encoder refined by contrastive learning and then clusters them into dynamically named types. Zero-shot typing has also been driven by the semantics of the type names themselves: one line of work leverages natural-language type *descriptions* [24] so that classes unseen at training time can still be recognised, a premise our label-name prototypes share. Progress in the low-resource regime is in turn anchored by dedicated benchmarks, notably Few-NERD [25], a large human-annotated dataset organised as a two-level hierarchy of 8 coarse and 66 fine-grained types. Other approaches require a general-purpose large language model to perform extraction directly, as in ChatIE [26], or focus on recovering entities that detectors tend to miss [27]. OPENER adopts the decoupled detect-then-type view of OWNER, but replaces the trained encoder and the unsupervised clustering with a pretrained, Matryoshka-truncatable embedding space and a balanced linear classifier, so that no encoder is trained from scratch.

C. LLM-based and Few-shot NER

A complementary line distils or instruction-tunes large language models for open NER. UniversalNER [4] distils a chat model into a smaller LLaMA-based student [28] over tens of thousands of entity types, while GoLLIE [5] conditions a code language model on annotation guidelines to follow unseen schemas. Few-shot methods reduce supervision further through prompt optimization [29], knowledge-graph-guided contrastive learning [30], or token-level contrastive objectives such as CONTaiNER [31]. These methods are accurate but rest on models with billions of parameters, whose inference cost, latency, and energy use are substantial, and which may not even fit on commodity hardware. At the small end of this family, however, a quantized instruction-tuned model does fit a commodity GPU: as a concrete, runnable reference point, we include Qwen2.5-1.5B-Instruct [6] in 4-bit, prompted to return entities as JSON, and report where it lands relative to the dedicated open-world systems (subsection IV-F). This cost is precisely what motivates the lightweight design and the energy-aware evaluation of OPENER.

D. Entity Representations and Contrastive Learning

OPENER relies on a pretrained sentence embedder rather than a task-specific encoder. Nomic Embed [14] provides an open, long-context model trained with the Matryoshka objective [15], whose representations can be truncated to lower dimensions without retraining, trading quality for compute. Such embedders are based on the Sentence-BERT line of work [32], and we sharpen entity-type geometry with a triplet margin objective in the spirit of FaceNet [33]. Refining a contrastive objective by mining and re-weighting hard negatives is a recurring recipe, as in GCSE [34], which down-weights false hard negatives with a Gaussian decay, close in spirit to our error-driven hard-negative mining. The choice of base encoder is made empirically:

we compare several strong open sentence encoders (E5 [13], BGE [10], MPNet [11] and mxbai [12]) on entity typing under a common protocol (Table IX). Nomic v1.5 [14] edges them out while additionally offering native Matryoshka truncation, so we adopt it as the base and fine-tune it. Embedding labels and entities in a shared space [35], modeling entity types with Gaussian components [36], generating synthetic entity-bearing text for augmentation [37], and using mixtures to augment training data [38] are recurring ideas in low-resource NER, all grounded in classical Gaussian mixture models fitted by expectation-maximization [39], [40]. Unlike methods that train a dedicated encoder, OPENER takes transfer learning in its purest form: it keeps a general-purpose pretrained space largely frozen and only applies a light contrastive fine-tuning, testing whether such a representation is already discriminative enough for entity typing.

E. Efficient and Green AI

A growing body of work argues that efficiency should be reported alongside accuracy. Prior work has quantified the financial and environmental cost of training NLP models [41], promoted efficiency as a first-class evaluation criterion under the *Green AI* agenda [42], and analysed the carbon footprint of machine-learning training and how to reduce it [43]. Complementary efforts argue for the systematic reporting of energy and carbon and provide a framework to track them per run [44], and take a whole-lifecycle, system-level view of AI's environmental footprint across data, algorithms, and hardware [45]. Most relevant to deployment, a study of *inference*-time cost [46] finds general-purpose generative models to be orders of magnitude more expensive per query than task-specific ones, precisely the gap a lightweight pipeline like ours seeks to close. Tools such as CodeCarbon [47] make per-run energy and carbon estimates practical. However, open-world NER systems are still compared almost exclusively on quality. Instead, OPENER reports a three-axis comparison of quality, measured by Adjusted Mutual Information [48], inference speed, and energy, placing the frugality of each method on an equal footing with its accuracy.

In summary, existing open-world NER methods either train a dedicated encoder [7] or query costly large language models [4], [5], and are compared on accuracy alone. OPENER fills both gaps: it reuses pretrained components with only a light contrastive step and a balanced linear classifier, and it is evaluated jointly on quality, speed and energy.

III. Method

A. Problem Formulation

Given a sentence, an open-world NER system must return a set of typed mentions whose type inventory is *not* fixed at training time. Formally, from a sentence s, a mention detector produces the set of candidate mentions $\mathcal{M}(s) = \{m_i\}$. Each mention m_i is then passed, together with its sentence s, to an embedder f_θ that maps it *in context* to a D-dimensional representation vector e_i. Finally, a typing head assigns m_i a predicted type $\hat{y}_i$ drawn from the type inventory $\mathcal{T}$,

$$e_i = f_\theta(s, m_i) \in \mathbb{R}^D, \qquad \hat{y}_i \in \mathcal{T}, \tag{1}$$

where the type inventory $\mathcal{T}$ is unknown at training time.

We adopt the same evaluation protocol as OWNER [7]. To keep the end-to-end score honest, detection errors must be penalised rather than quietly ignored. We align predicted and gold mentions by their exact character offsets, so a prediction is a match only when it covers exactly the same span as a gold mention. Any mention left unmatched is then a detection error of one of two kinds. A gold mention that no prediction matches is a *false negative* (a missed entity, for example, the detector finds "Marie Curie" but misses "radium" in "Marie Curie discovered radium"). A predicted mention that matches no gold entity is a *false positive* (a spurious span, for example, the detector wrongly proposes "discovered" as an entity). Each unmatched mention receives its own distinct sentinel type, so that these errors count as mismatches when computing the AMI instead of being ignored.

Type-assignment quality is then measured by Adjusted Mutual Information (AMI) [48] between the gold type partition U and the predicted partition V over the aligned mentions,

$$\mathrm{AMI}(U, V) = \frac{\mathrm{MI}(U, V) - \mathbb{E}[\mathrm{MI}(U, V)]}{\frac{1}{2}[H(U) + H(V)] - \mathbb{E}[\mathrm{MI}(U, V)]}, \tag{2}$$

where MI is the mutual information and H the entropy, and the denominator uses the arithmetic-mean normalisation (the scikit-learn default). Intuitively, mutual information is a core quantity of Shannon's information theory [49]: it measures how much knowing a mention's predicted type reduces the uncertainty about its true (gold) type, and it is zero exactly when the two partitions are statistically independent (the prediction tells us nothing about the truth). The *adjusted* form further subtracts the agreement expected by chance, so a random typing scores close to 0 whatever the number of types, while a perfect one scores 1. A key advantage in the open-world setting is that AMI does not require the two label vocabularies to coincide, since the predicted type names need not match the gold ones.

We report two protocols that are never mixed in the same comparison: *typing-on-gold*, where the typing component is applied to gold spans and detection is bypassed, and *end-to-end*, where mentions come from the detector and the sentinels above are active.

B. Architecture Overview

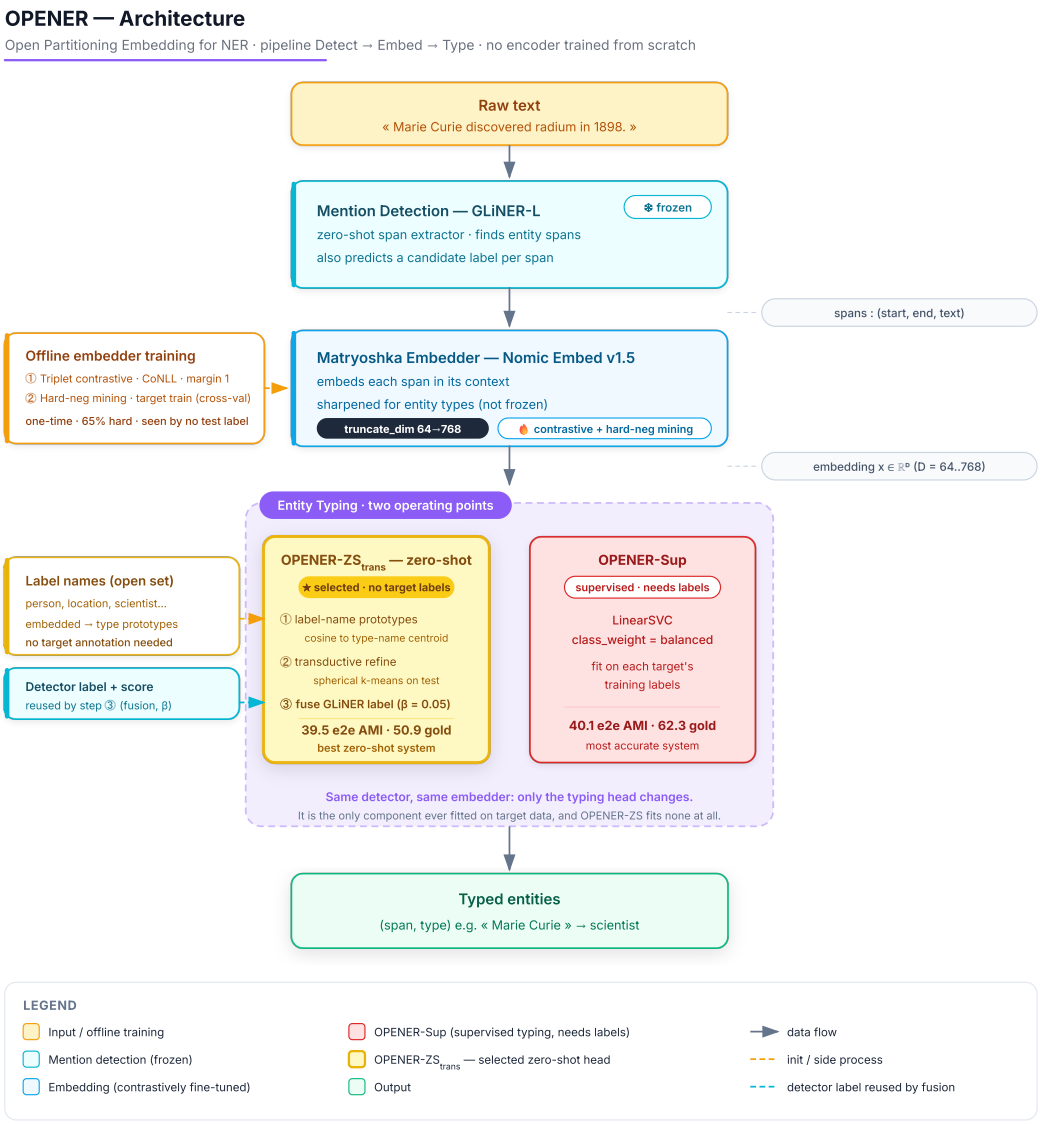

Fig. 1. The OPENER pipeline (Detect → Embed → Type): a frozen GLiNER detector proposes spans, a Matryoshka embedder (Nomic, contrastively fine-tuned) maps each span in context to a vector, and a typing head assigns a type. The typing head (the only target-fitted component) has two operating points: the *zero-shot* OPENER-ZS$_{\text{trans}}$(★, selected) and OPENER-Sup (*supervised*, the upper bound). All upstream components are ready-to-use.

OPENER is a three-stage pipeline, Detect → Embed → Type (Figure 1), assembled from ready-to-use components so that no encoder is trained from scratch. Three principles guide the design.

- A pretrained detector and a pretrained sentence embedder are reused as-is.
- A light contrastive fine-tuning, refined by error-driven hard-negative mining, turns the general-purpose embedding into one that is discriminative for entity *types*.
- The embedder is Matryoshka-truncatable, exposing a compute/quality knob.

Only the typing head is fitted on target-side material, and it comes in two forms: a zero-shot form, OPENER-ZS, that uses no target labels at typing time and OPENER-Sup, a supervised linear probe that is the strongest configuration but requires target labels.

1. Mention Detection

Mention detection is delegated to GLiNER [2], a frozen, ready-to-use detector that scores every span against the candidate type names in a single forward pass. Decoupling detection from typing lets each component be chosen and measured independently, and it lets the typing head operate on any span proposer. We use GLiNER-L by default, as it yields the best AMI. The small and medium variants trace a size/efficiency trade-off (see Table IX for the full detector comparison). As shown in subsection IV-F, the detector is the shared end-to-end bottleneck: its recall on specialised domains, not the typing head, caps end-to-end quality.

2. Entity Embedding

Detected spans are embedded with Nomic Embed v1.5 [14], a sentence embedder trained with the Matryoshka objective [15], whose representation can be truncated to a lower dimension and renormalised without retraining. Each mention is embedded *in context* rather than in isolation: the span is presented together with its surrounding sentence, so that polysemous surface forms (e.g. Apple as a company versus a fruit) are disambiguated. This yields a vector $e_i \in \mathbb{R}^D$ with D truncatable in $\{768, 512, 256, 128, 64\}$. We keep the full D=768, which gives the best AMI (see Table IX for the full dimension sweep). Truncation keeps only the first d coordinates of the embedding ($d \leq D$) and rescales them back to unit length,

$$e_i^{(d)} = e_{i,1:d} \,/\, \|e_{i,1:d}\|_2, \tag{3}$$

where $e_i \in \mathbb{R}^D$ is the in-context embedding of mention m_i, $e_{i,1:d}$ is its truncation to the first d components (the coordinates $d{+}1$ to D are simply dropped), and $\|e_{i,1:d}\|_2 = \sqrt{\sum_{k=1}^{d} e_{i,k}^2}$ is its Euclidean (L_2) norm, that is the geometric length of the vector. Dividing by this norm rescales $e_i^{(d)}$ to length 1, so that embeddings truncated at different d remain comparable through cosine similarity. This shrinks the stored embedding and the typing-head fit cost but leaves the forward pass unchanged (Table IX).

Still, the frozen embedding is only weakly discriminative for entity types (25.8 AMI, Table IX), so we sharpen its geometry with *contrastive fine-tuning*, in two stages, while keeping the backbone otherwise intact.

a) Stage 1: contrastive: We fine-tune the embedder with a Triplet Margin Loss $\mathcal{L}$ [33] (using sentence-transformers [32]), with margin $\alpha = 1$, on the CoNLL-2003 [20] training spans. We use CoNLL-2003 because its generic, domain-neutral label set makes it the standard source domain of our benchmark (Table I). Since the embedder is only lightly fine-tuned on it, we exclude CoNLL-2003 from the reported average, so as not to bias the results in our favour, following the same protocol as OWNER. A triplet (a, p, n) takes an anchor span a, a positive p of the *same* type, and a negative n of a *different* type, with at most two triplets per anchor. Writing α for the margin, each triplet contributes

$$\mathcal{L}(a, p, n) = \max\big(0,\ \|e_a - e_p\|_2 - \|e_a - e_n\|_2 + \alpha\big), \quad \alpha = 1, \tag{4}$$

which pulls same-type spans together and pushes different-type spans apart, i.e. it decreases the embedding distance $\|e_a - e_p\|_2$ while increasing $\|e_a - e_n\|_2$. Training runs for 3 epochs with batch size 16, learning rate 2×10^{-5} and 100 warmup steps. This single step lifts typing-on-gold AMI from 25.8 to 54.0. The UMAP projection in Figure 6 shows how Stage 1 reshapes the embedding space.

b) Stage 2: error-driven hard-negative mining: We then mine the model's own confusions. We defer this to Stage 2, after the Stage-1 warm-up, to avoid overfitting to these hard cases. On each target training split, we run a cross-validated typing pass (so no test label is ever seen) and collect the type pairs that are most often confused, such as *writer*/*person*, *album*/*band* and *country*/*location*. For each such error, we form a *hard* triplet whose anchor is the mistyped span (true type L), whose positive is another span of L, and whose negative is a span of the confused type C. Hard triplets are mixed with correctly-typed (easy) triplets for diversity, targeting a 65% hard fraction, and the Stage-1 embedder is fine-tuned further with the same optimiser settings. This ratio must be controlled carefully to preserve generalisation. The selected run mines 8000 triplets over 3 epochs and reaches 62.5 AMI at the released seed (+8.5 over Stage 1, improving on 12 of the 13 sets). Retraining the full two-stage recipe under three seeds yields 62.3 ± 1.0 (subsection IV-D). This embedder is shared unchanged by every typing head below. The UMAP projection in Figure 6 also reveals the additional effect of Stage 2 on the embedding space.

3. Entity Typing

The embedding above feeds a typing head, the only component fitted on target-side material. It has two operating points, presented in turn: a label-free *zero-shot* head, OPENER-ZS, which we adopt as the open-world default, and a *supervised* probe, OPENER-Sup, a stronger but label-dependent upper bound.

a) Zero-shot typing (OPENER-ZS): To remove target supervision at typing time, each type t is represented by a prototype p_t: the ℓ_2-normalised centroid of the embeddings of its name (its anchor words) in the contrastive space,

$$p_t = \bar{e}_t \,/\, \|\bar{e}_t\|_2, \qquad \bar{e}_t = \frac{1}{|\mathrm{name}(t)|} \sum_{w \,\in\, \mathrm{name}(t)} f_\theta(w), \tag{5}$$

where t is a candidate type, $\mathrm{name}(t)$ is the set of words forming its name (its anchor words, e.g. "person" or "astronomical object"), $f_\theta(w)$ is the embedding of anchor word w in the contrastive space, and $\bar{e}_t$ is their average (the centroid), which dividing by its Euclidean norm $\|\bar{e}_t\|_2$ turns into a unit-length prototype p_t (the averaging factor cancels under this normalisation). A mention m_i is then assigned the type of the nearest prototype by cosine similarity, $\hat{y}_i = \arg\max_t \cos(e_i, p_t)$. This inductive nearest-centroid head is OPENER-ZS$_{\text{ind}}$. Two label-free refinements improve this head without using any target label. First, a *transductive* refinement runs one pass of spherical k-means over the unlabelled test mentions, seeded by the prototypes. Second, a *detector fusion* reuses a signal the prototype head otherwise discards: GLiNER already predicts a zero-shot type for each span with confidence g_i, which we add to the similarity score before taking the arg-max,

$$s_t(i) = \cos(e_i, p_t) + \beta\, g_i \mathbb{1}[t = \ell_{\text{GLiNER}}(i)]\,, \qquad \hat{y}_i = \arg\max_t s_t(i), \tag{6}$$

where $s_t(i)$ is the fused score of assigning type t to mention m_i, $\cos(e_i, p_t)$ is the prototype similarity above, $g_i \in [0, 1]$ is GLiNER's own confidence for the type $\ell_{\text{GLiNER}}(i)$ it predicted, and $\mathbb{1}[\cdot]$ is the indicator function, equal to 1 when t is exactly that predicted type and 0 otherwise. The fusion weight $\beta{=}0.05$ was found by a small empirical sweep and is deliberately kept low, so the detector only nudges the arg-max, and the assigned type $\hat{y}_i$ is the highest-scoring one. The refined, fused head is OPENER-ZS$_{\text{trans}}$, the selected zero-shot operating point: it reaches 39.5 AMI end-to-end (mean over three retraining seeds), ahead of every compared zero-shot system, while using no target labels (subsection IV-F).

b) Supervised probe (OPENER-Sup): When a domain's entity classes are fixed and labelled, the typing head can instead be a balanced linear classifier (LinearSVC with the LIBLINEAR solver [50]) fitted one-vs-rest on the target's training labels. For each type c, it minimises a squared-hinge objective with balanced class weights,

$$\min_{w_c, b_c} \; \tfrac{1}{2}\|w_c\|_2^2 + C \sum_i \omega_{y_i} \, \max\!\big(0,\, 1 - z_{ic}(w_c^\top e_i + b_c)\big)^2, \quad \omega_c = \frac{n}{|\mathcal{T}|\, n_c}, \tag{7}$$

where, for each type c, w_c and b_c are the weight vector and bias of its one-vs-rest hyperplane, e_i is the (frozen) embedding of training mention i and y_i its gold type, $z_{ic}{=}{+}1$ if $y_i{=}c$ and -1 otherwise, and C is the regularisation strength that trades margin width against training errors. The squared-hinge term $\max(0,\, 1 - z_{ic}(w_c^\top e_i + b_c))^2$ penalises mentions on the wrong side of the margin. The balancing weight $\omega_c = n/(|\mathcal{T}|\, n_c)$, with n the total number of training mentions, $|\mathcal{T}|$ the number of target types and n_c the count of type c, up-weights rare types to counter the benchmark's heavy imbalance. A mention is finally typed as $\hat{y}_i = \arg\max_c(w_c^\top e_i + b_c)$. This is the strongest configuration but requires target-side labels.

C. Evaluation Protocol

We evaluate on the 13 datasets of subsection IV-A, each test split capped at 1000 sentences. Quality is AMI [48] (with macro-F_1 and accuracy), latency is the median (p50) per-sentence time over GPU-synchronised, warmed-up passes, and energy is measured with CodeCarbon [47] under the French grid carbon intensity. The full protocol and instrumentation are detailed in subsection IV-D and subsection IV-E. The code and the per-experiment configurations are available on GitHub at [ANONYMIZED FOR REVIEW], and the fine-tuned embedders of OPENER-ZS and OPENER-Sup are available on Hugging Face at [ANONYMIZED FOR REVIEW], respectively.

IV. EXPERIMENTS

A. Datasets

TABLE I
THE 13 EVALUATION SETS AND THEIR GOLD ENTITY TYPES.

Dataset	Abbr.	Domain	Test	Entity types (share %)
CrossNER-AI	AI	encyclopaedic (AI)	430	task 12, field 11, product 11, metrics 11, misc 10, algorithm 10, researcher 9, organisation 8, conference 5, person 4, programlang 3, country 2, location 2, university 2
CrossNER-Literature	Lit	encyclopaedic (literature)	416	writer 25, book 18, misc 11, literarygenre 9, person 8, award 6, poem 5, organisation 5, country 4, location 4, magazine 3, event 2
CrossNER-Music	Mus	encyclopaedic (music)	465	musicalartist 15, musicgenre 15, band 14, location 10, album 10, award 8, song 7, organisation 6, misc 5, country 5, person 2, event 2, musicalinstrument 1
CrossNER-Politics	Pol	encyclopaedic (politics)	651	politicalparty 23, location 14, organisation 12, politician 12, election 10, country 10, person 8, misc 6, event 5
CrossNER-Science	Sci	encyclopaedic (science)	543	misc 17, scientist 15, astronomicalobject 11, organisation 9, person 7, academicjournal 6, location 6, chemicalcompound 5, protein 5, award 5, university 4, chemicalelement 3, enzyme 3, discipline 2, event 1, country 1, theory 1
WNUT-17	WNUT	Social media	689	person 40, group 15, location 14, creative-work 13, product 12, corporation 6
MIT-Restaurant	Rest	Restaurant search	1000	Location 21, Dish 18, Amenity 16, Hours 15, Restaurant_Name 11, Price 10, Cuisine 5, Rating 4
MIT-Movie	Movie	Movie trivia	1000	TITLE 19, ACTOR 16, YEAR 16, GENRE 16, RATING 9, CHARACTER 9, TRAILER 5, REVIEW 3, RATINGS_AVERAGE 3, PLOT 2, DIRECTOR 2, SONG 1
FabNER	Fab	Manufacturing	1000	MATE 27, CONPRI 21, MANP 15, PRO 11, APPL 6, FEAT 6, PARA 4, MACEQ 3, BIOP 2, CHAR 2, ENAT 2, MANS <1
BioNLP/JNLPBA-2004	Bio	Biomedical	1000	protein 53, cell_type 22, DNA 13, cell_line 10, RNA 1
CoNLL-2003	CoNLL	News	1000	PER 32, ORG 29, LOC 28, MISC 11
GUM	GUM	Multi-genre	1000	abstract 38, person 17, event 12, place 10, object 8, organization 8, time 5, substance 1, animal 1, plant <1
GENTLE	GEN	Multi-genre (challenge)	1000	abstract 55, person 10, event 9, time 8, object 7, place 5, organization 4, animal 1, substance 1, plant <1

"Test" column is the number of test sentences used (capped at 1000). Each type is followed by its share (%) of the test mentions, listed by decreasing frequency (types are the dataset's original labels).

We evaluate on 13 evaluation sets that span news, social media, spoken queries, encyclopaedic text, biomedical abstracts and manufacturing literature, following the dataset selection of OWNER [7]. CrossNER [51] contributes five domain-specific sets, evaluated separately (artificial intelligence, literature, music, politics, science). The remaining eight are CoNLL-2003 [20], WNUT-17 [52], the MIT-Restaurant and MIT-Movie spoken-query corpora [53], FabNER [54], the BioNLP/JNLPBA-2004 biomedical task [55], and the GUM [56] and GENTLE [57] multi-genre corpora. Label granularity ranges from four coarse types in CoNLL-2003 to seventeen fine-grained categories in CrossNER-Science, including the opaque domain acronyms of FabNER. Two datasets used by OWNER [7], GENIA [58] and the 2014 i2b2/UTHealth clinical corpus, are license-gated and therefore not re-measured here. BioNLP/JNLPBA-2004 serves as our biomedical proxy. To homogenise evaluation cost across systems, we cap each test split at 1000 sentences (the full split is used when it is smaller).

B. Baselines

TABLE II
BACKBONES AND SIZE OF THE RE-MEASURED BASELINES AND OPENER, IN THE SPIRIT OF OWNER [7].

Model	Backbone	Type	Version	Params	vs OPENER
GLiNER-S	DeBERTaV3	Encoder	deberta-v3-small	50M	0.1×
GLiNER-M	DeBERTaV3	Encoder	deberta-v3-base	200M	0.4×
GLiNER-L	DeBERTaV3	Encoder	deberta-v3-large	330M	0.7×
GNER	T5	Enc.–Dec.	t5-base	220M	0.5×
OWNER†	DeBERTaV3+BERT	Encoder	deberta-v3 + bert-base	294M	0.6×
Qwen2.5-1.5B	Qwen2.5	Decoder	qwen2.5-1.5b-instruct	1.5B	3.2×
OPENER (ours)	DeBERTaV3+Nomic	Encoder	deberta-v3-large + nomic-v1.5	467M	1.0×

"Type" column is the transformer family. "vs OPENER" column is the total parameter count relative to OPENER's end-to-end pipeline (467M = GLiNER-L 330M detector + Nomic 137M embedder).
† OWNER reports 110M for its BERT typing encoder alone. We count detection + typing for parity with OPENER.

We compare OPENER with representative open-world systems, all run end-to-end on the same splits and scored with the same protocol (subsection IV-D). Table II summarises their backbones and sizes.

- **GLiNER** [2]: an open-world span classifier that scores every candidate span against the supplied type names in a single forward pass on a DeBERTaV3 encoder [22]. It is a strong and fast detector, but its predicted types are limited to the names given as queries. We run all three sizes (S/M/L) to trace a size/efficiency trade-off.
- **GNER** [3]: a generative tagger built on T5 [23] that regenerates the sentence with inline type tags. It is accurate, but its autoregressive decoding makes inference markedly slower than the encoder-based detectors. We run the T5-base checkpoint.
- **OWNER** [7]: our closest baseline, a DeBERTaV3 detector with a BERT typing encoder [8] refined by contrastive learning, whose mentions are then grouped into dynamically named types by a plain unsupervised k-means. This keeps it fully label-free, but it re-encodes and re-clusters every mention at inference time, which is costly. We execute it in its original environment.
- **Qwen2.5-1.5B-Instruct** [6]: the only generative LLM that fits our 6 GB GPU, runs in 4-bit NF4 [59] and prompted for JSON-formatted entities. Even at this small size, its autoregressive decoding makes it the slowest system we measure and, per sentence, the most energy-hungry, showing that generative extraction remains costly regardless of model size.
- **Large LLM systems**: UniversalNER [4], GoLLIE [5] and ChatIE [26] reach strong accuracy but rely on 7B-parameter backbones [28] whose compute and energy cost is far higher and which exceed our hardware even under 4-bit quantisation. We therefore discuss them qualitatively and omit them from Table II and the measured comparison. For LLM extraction figures under a comparable AMI protocol, we refer the reader to the benchmark of OWNER [7], whose findings are consistent with our, measurements.

C. Metrics

Our primary metric is the Adjusted Mutual Information (AMI) [48] between predicted and gold type assignments, following OWNER [7], complemented by macro-averaged F_1 and accuracy. Because open-world predictions need not share the gold label vocabulary, AMI is computed over the aligned set of mentions, which makes it well suited to comparing dynamically named types. Beyond quality, we report two efficiency axes: inference *speed*, as the median (p50) latency per sentence, and *energy*, as kilowatt-hours and grams of CO_2eq per run.

D. Evaluation Protocol

The predicted and gold mentions are aligned by exact character offsets, and any unmatched mention receives a distinct sentinel label so that detection errors are penalised rather than ignored, following the sentinel-based scoring detailed in subsection III-A. We distinguish two protocols that must not be mixed in the same comparison. In the *typing-on-gold* protocol, the entity-typing component is applied to gold mention spans, which measures typing quality in isolation under perfect detection. In the *end-to-end* protocol, the detector produces mentions and the sentinel labels above are active, which is a setting comparable to the baselines. OPENER's embedder *training* is stochastic (triplet sampling and optimisation), so we retrain the complete embedder (contrastive Stage 1, then hard-negative mining Stage 2) under three random seeds (7, 42, 123) and re-evaluate every OPENER operating point on the full benchmark with each resulting embedder. Therefore, every OPENER cell in the result tables reports the mean ± standard deviation over the three seeds. Seed 42 is the model released. The compared baselines are frozen pretrained checkpoints whose inference is deterministic, so re-running them under a different seed returns identical predictions and they are reported as single runs. OWNER, the only baseline that trains, is likewise reported from a single training run of its published

TABLE III
TEST-SET BOOTSTRAP ROBUSTNESS OF THE RELEASED (SEED-42) MODEL ON THREE REPRESENTATIVE DATASETS.

Model	CoNLL-2003	CrossNER-Music	FabNER
OPENER-ZS$_{\text{ind}}$	66.1 ± 1.4 [63.5, 69.0]	55.5 ± 0.9 [53.8, 57.3]	36.2 ± 0.7 [34.7, 37.5]
OPENER-Sup	71.7 ± 1.5 [68.9, 74.6]	72.4 ± 0.9 [70.6, 74.2]	59.8 ± 0.9 [57.9, 61.6]

For OPENER-Sup and the inductive zero-shot head OPENER-ZS$_{\text{ind}}$ (the transductive refinement is excluded, as it adapts to the resampled test distribution). Cells report typing-on-gold AMI (↑) as bootstrap mean ± std with the 95% percentile CI (K=1000 resamples of the gold mentions). Point estimates on the full test set are 71.5/71.8/59.2 (Sup) and 65.9/55.2/35.6 (OPENER-ZS$_{\text{ind}}$), matching the bootstrap means. The bootstrap std stays at or below 1.5 AMI, indicating stable scores.

TABLE IV
OPENER CONFIGURATION.

Component	Setting
Mention detector	GLiNER-L (frozen) [2]
Entity embedder	Nomic v1.5, D=768, span-in-context [14]
Contrastive loss	Triplet margin 1 [33]
source	CoNLL-2003 train spans [20]
optimiser	3 epochs, batch 16, lr 2×10^{-5}, 100 warmup
Hard-neg. mining	8000 triplets, 65% hard, 3 epochs
source	target train splits, cross-validated
Supervised typing	LinearSVC, balanced weights [50]
Zero-shot typing	label-name prototype + transductive + fusion (β=0.05)
Random seeds	7 / 42 / 123 (full embedder retraining per seed; 42 = released)

The detector and embedder are ready-to-use. The contrastive and hard-mining stages produce the shared embedder. The typing head is fitted (supervised) or label-free (zero-shot).

protocol. We also quantify the test-set sampling uncertainty with a bootstrap on the released model (Table III), a source of uncertainty orthogonal to the retraining seed.

E. Experimental Settings

All experiments run on our single consumer-grade GPU with 6 GB of VRAM (CUDA 12.1, PyTorch 2.5.1 [60]). The goal of using such modest hardware was to verify that the models could also run on standard personal computers, demonstrating that our approach is highly versatile. We use the HuggingFace Transformers library [61] for baselines, sentence-transformers [32] for the Matryoshka embedder [14], [15], and scikit-learn [62] for the linear classifier, whose solver is LIBLINEAR [50]. Latency is measured with two warmup passes followed by timed passes with GPU synchronisation, reporting p50, p95 and p99 per sentence. Energy is measured with CodeCarbon [47] in offline mode using the French grid carbon intensity. When hardware power counters are unavailable, we fall back to a thermal-design-power estimate and flag the measurement method in the report. All systems are measured on the same machine. Our consumer GPU is subject to *thermal throttling* (it lowers its clock frequency under sustained load to stay within its temperature limit), so per-sentence latency and per-run energy carry roughly $\pm 15\%$ run-to-run variance. Therefore, we read the per-system averages as indicative and do not over-interpret individual per-dataset cells. The quality metrics, in contrast, are fully deterministic: with the fixed seed and the released model, no stage of the OPENER pipeline is stochastic at inference, so its AMI and F_1 reproduce exactly across re-runs (we verified identical CrossNER scores on an independent re-run), and only latency and energy fluctuate. Because inference is deterministic, the only remaining source of uncertainty in quality scores is the finite test set. We quantify it with a non-parametric bootstrap: for three representative datasets, we resample the gold mentions with replacement (K=1000 resamples) and recompute typing-on-gold AMI, yielding the 95% confidence intervals in Table III. The uncertainty is small (bootstrap std of at most 1.5 AMI, 95% CIs within roughly ± 3 AMI) and the bootstrap means coincide with the released model's full-test-set scores (given in the caption of Table III), indicating that the reported figures are stable rather than fragile seed-specific artefacts. Table IV collects the complete OPENER configuration used throughout. The embedder is retrained under each of the three seeds (7, 42, 123) while every other component is fixed.

F. Main Results

The tables below report the benchmark on all 13 sets, and Figure 2 plots the per-dataset end-to-end AMI of every system. OWNER is run in its native protocol [7]: a single model is trained once on the CoNLL-2003 source (a one-time cost of 88 minutes on our 6 GB GPU) and then applied *zero-shot* to every target set (cells marked §). CoNLL-2003 is therefore in-domain

for OWNER (marked †) and is excluded when reading OWNER as a transfer system. Concretely, excluding CoNLL-2003 (in-domain for both OWNER and OPENER), OWNER's reported 34.3 thirteen-set mean becomes a 32.9 transfer mean, still well below OPENER-ZS$_{\text{trans}}$(39.1 on the same 12 sets). The margin in fact widens from $+5.2$ to $+6.2$ AMI under this fair transfer reading. One protocol asymmetry must be kept in mind when reading the typing scores: OPENER-Sup fits a lightweight linear classifier on each target's training split (a supervised probe), whereas OWNER, GLiNER and GNER use no target-side labels. The two systems thus trade annotation for inference cost: OPENER-Sup needs target labels but stays cheap at test time, while OWNER needs none but pays a heavy per-sentence cost, it re-encodes every mention and clusters at inference time, an order of magnitude above OPENER (Table VI, Table VII). To close this asymmetry, we also evaluate OPENER-ZS, a zero-shot variant that drops the supervised probe and types each mention against label-name prototypes, using no target labels. Its un-refined nearest-centroid baseline, OPENER-ZS$_{\text{ind}}$, already sits within half a point of the zero-shot OWNER baseline end-to-end (33.8 vs 34.3 mean AMI) and matches it on gold (43.2 vs 43.0, Table X), while keeping OPENER's frugal inference profile (137 ms, 1.85 Wh vs 616 ms, 14.3 Wh). Two label-free refinements then build OPENER-ZS$_{\text{trans}}$: a transductive k-means on the prototypes, and *fusing in the detector's own zero-shot label predictions*, a signal the centroid otherwise discards, which together reach 39.5 end-to-end, the best of all compared zero-shot systems, ahead of GLiNER-L (38.9) and within 0.6 AMI of OPENER-Sup, all without any target label. For reference, the only generative LLM that fits our 6 GB GPU, Qwen2.5-1.5B-Instruct [6] in 4-bit, prompted for JSON-formatted entities, trails every dedicated system (26.1 mean AMI on a 150-sentence subsample, Table V) while being the slowest per sentence (5.1 s) and the most energy-hungry once normalised by sentence count (11.4 Wh over only 150 sentences, about twice GNER's per-sentence draw, Table VIII): a small open LLM pays on both axes at once. OPENER-Sup remains the stronger configuration (40.1 end-to-end, 62.3 on gold), so OPENER-ZS is best read as the frugal, annotation-free operating point of the same pipeline. A per-domain analysis of these results, the detection bottleneck and the learned embedding geometry follows in subsection IV-G.

TABLE V
OPEN-WORLD NER, END-TO-END. AMI ($\times 100$, $\uparrow$).

Model	AI	Lit	Mus	Pol	Sci	WNUT	Rest	Movie	Fab	Bio	CoNLL	GUM	GEN	Avg
Zero-shot (no target labels)														
GLiNER-S	41.7	49.8	56.4	50.6	53.3	29.7	26.1	35.6	10.7	31.8	18.9	**34.1**	**36.1**	36.5
GLiNER-M	42.0	**50.1**	57.4	50.2	**54.2**	28.1	27.9	36.4	11.3	34.9	24.9	32.8	31.6	37.1
GLiNER-L	42.9	48.9	58.0	49.8	52.8	28.9	30.9	37.6	28.1	34.6	29.7	32.7	30.5	38.9
GNER	41.9	46.9	53.5	48.9	49.9	**30.0**	30.0	36.8	17.0	29.1	**43.7**	32.8	31.6	37.9
Qwen2.5-1.5B‡	25.4	30.5	30.5	28.3	34.1	27.8	5.0	27.4	12.9	29.9	15.5	37.2	34.6	26.1
OWNER	**43.9**	44.0	46.6	**50.9**	50.3	22.3	7.6	24.0	18.7	26.5	50.3†	29.1	31.3	34.3
OPENER-ZS$_{\text{ind}}$	$29.7_{\pm 2.2}$	$41.6_{\pm 2.3}$	$43.6_{\pm 0.3}$	$40.4_{\pm 0.7}$	$42.5_{\pm 0.6}$	$21.9_{\pm 1.3}$	$28.0_{\pm 1.9}$	$32.8_{\pm 0.9}$	$27.6_{\pm 0.1}$	$26.2_{\pm 1.4}$	$43.5^{\dagger}_{\pm 1.6}$	$30.8_{\pm 1.4}$	$30.1_{\pm 1.0}$	$33.8_{\pm 0.7}$
OPENER-ZS$_{\text{trans}}$	$39.4_{\pm 2.1}$	$48.7_{\pm 0.7}$	**58.3**$_{\pm 0.3}$	$49.2_{\pm 0.9}$	$52.6_{\pm 0.2}$	$28.8_{\pm 1.0}$	$31.2_{\pm 0.2}$	$36.0_{\pm 0.6}$	$28.1_{\pm 0.3}$	$34.7_{\pm 0.9}$	$43.7^{\dagger}_{\pm 1.1}$	$32.0_{\pm 0.7}$	$30.7_{\pm 0.3}$	$39.5_{\pm 0.1}$
Supervised (target labels)														
OPENER-Sup	$40.3_{\pm 0.2}$	$48.7_{\pm 1.3}$	$55.2_{\pm 0.8}$	$49.1_{\pm 1.4}$	$51.8_{\pm 1.7}$	$24.8_{\pm 0.2}$	**34.1**$_{\pm 0.9}$	**37.8**$_{\pm 0.2}$	**29.8**$_{\pm 0.5}$	**35.9**$_{\pm 0.5}$	$46.1^{\dagger}_{\pm 2.1}$	$33.8_{\pm 0.1}$	$34.0_{\pm 0.3}$	**40.1**$_{\pm 0.5}$

Abbreviations follow Table I. Bold marks the best value per column across all systems. Systems are grouped by supervision (*zero-shot* vs the *supervised* OPENER-Sup). OPENER rows report mean ± std over the three embedder-retraining seeds (subsection IV-D). The baselines are frozen checkpoints with deterministic inference (single run). Typing-on-gold is reported in Table X.
† CoNLL-2003 is in-domain for OWNER (training source) and OPENER (contrastive source) and is not bolded as a transfer result.
‡ Qwen2.5-1.5B-Instruct (4-bit) is on a 150-sentence subsample (indicative).

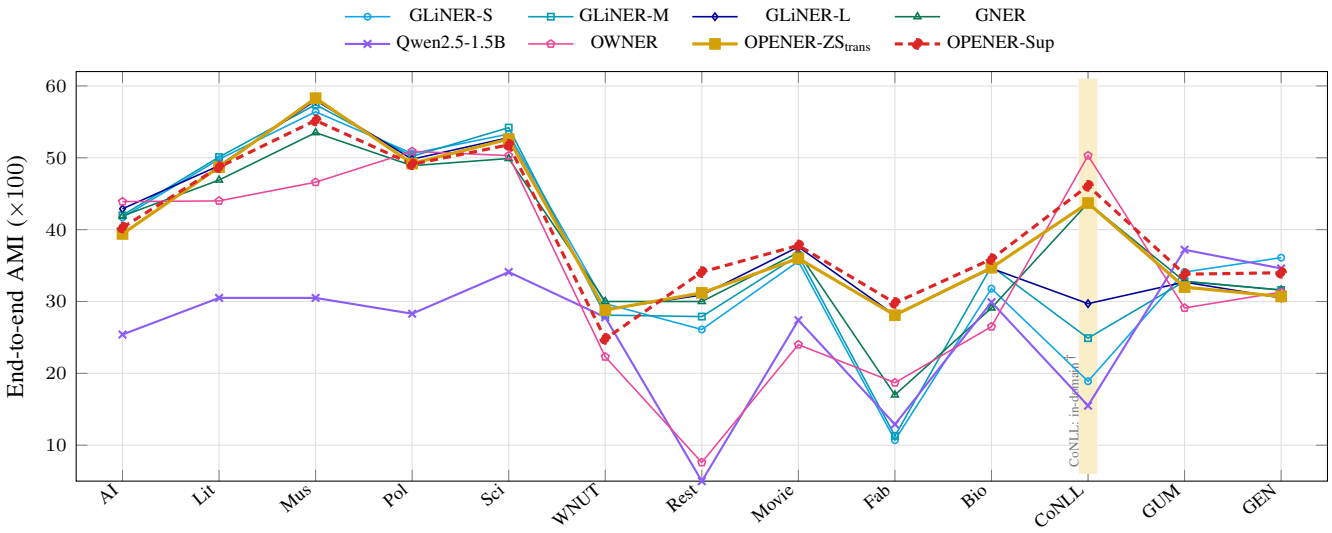

Fig. 2. End-to-end AMI per dataset (Table V, OPENER curves show the three-seed means). *Zero-shot* systems are drawn solid. OPENER-Sup (the *supervised* upper bound) is the dashed red line. † CoNLL-2003 (shaded) is in-domain for OWNER and OPENER. ‡ Qwen is evaluated on a 150-sentence subsample per set.

TABLE VI
MEDIAN PER-SENTENCE LATENCY (MS, ↓), END-TO-END.

Model	AI	Lit	Mus	Pol	Sci	WNUT	Rest	Movie	Fab	Bio	CoNLL	GUM	GEN	Avg
Zero-shot (no target labels)														
GLiNER-S	**21**	**22**	**21**	**21**	**21**	**20**	**19**	**20**	**20**	**22**	**20**	**21**	**21**	**21**
GLiNER-M	36	36	37	38	39	39	38	40	41	42	44	46	45	40
GLiNER-L	144	149	153	149	155	137	132	145	147	143	131	141	137	143
GNER	3431	5408	7461	6472	7110	2906	1610	3344	4310	5034	2914	3435	1500	4226
Qwen2.5-1.5B‡	4857	6757	9396	7700	8261	3553	2142	5014	2921	4737	2260	4265	3806	5051
OWNER§	500	712	895	920	757	311	281	532	438	558	719†	759	626	616
OPENER-ZS$_{ind}$	109	135	156	172	175	136	127	137	133	136	117	127	124	137
OPENER-ZS$_{trans}$	218	254	291	277	289	185	173	205	84	98	92	104	97	182
Supervised (target labels)														
OPENER-Sup	222	271	293	159	132	100	92	98	104	108	89	100	94	143

Abbreviations follow Table I, best per column in bold.
§ OWNER exposes no per-sentence call: its figure is an amortised wall-clock mean (its 88-min training is excluded).
† CoNLL is OWNER's in-domain source, measured by a separate inference-only run.
‡ Qwen2.5-1.5B-Instruct (4-bit) is on a 150-sentence subsample per set.

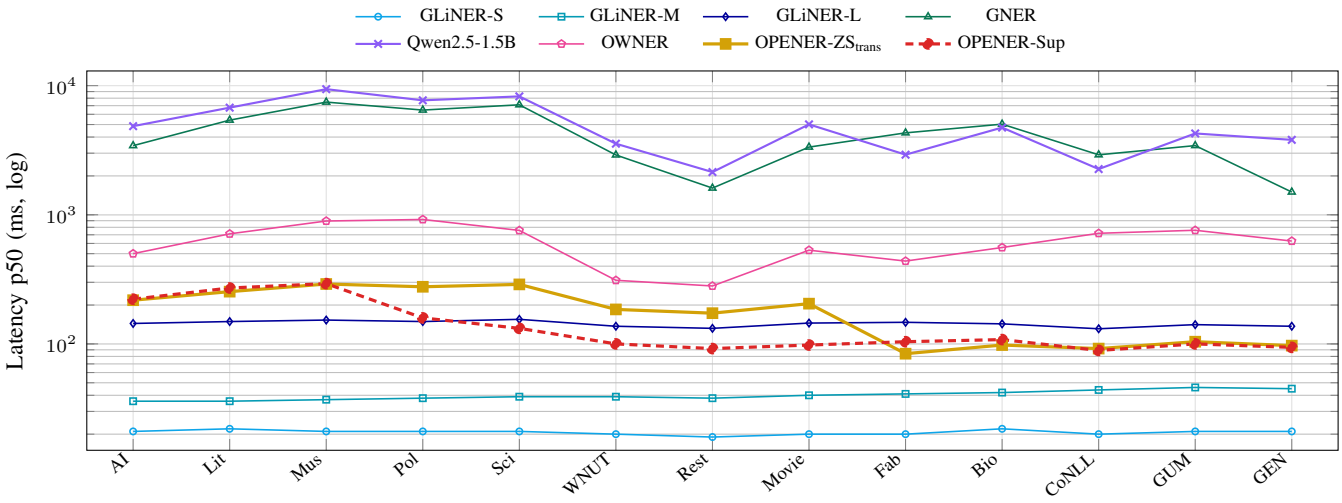

Fig. 3. Per-sentence inference latency (p50, ms, *log* scale) per dataset (Table VI). Same systems and styling as Figure 2. OPENER-Sup is the dashed red line. The OPENER variants track GLiNER-L, while GNER and the Qwen LLM are one to two orders of magnitude slower.

TABLE VII
ENERGY CONSUMPTION PER RUN (WH, ↓), END-TO-END.

Model	AI	Lit	Mus	Pol	Sci	WNUT	Rest	Movie	Fab	Bio	CoNLL	GUM	GEN	Avg
Zero-shot (no target labels)														
GLiNER-S	**0.19**	**0.17**	**0.20**	**0.27**	**0.22**	**0.27**	**0.35**	**0.39**	**0.40**	**0.42**	**0.37**	**0.41**	**0.40**	**0.31**
GLiNER-M	0.31	0.29	0.33	0.48	0.40	0.50	0.66	0.76	0.77	0.79	0.69	0.82	0.75	0.58
GLiNER-L	0.68	0.67	0.77	1.04	0.91	1.01	1.37	1.51	1.54	1.49	1.38	1.52	1.43	1.18
GNER	17.13	24.28	39.75	50.88	44.76	20.60	17.02	37.17	49.11	63.50	45.22	38.69	26.50	36.51
Qwen2.5-1.5B‡	12.7	15.1	17.5	18.4	18.0	7.9	4.6	9.5	7.0	11.1	7.3	10.2	9.5	11.4
OWNER§	6.6	9.0	12.7	18.3	12.6	6.5	8.6	16.3	13.4	17.1	22.0†	23.2	19.1	14.3
OPENER-ZS$_{ind}$	0.95	1.06	1.37	1.97	1.69	1.52	2.07	2.28	2.15	2.30	2.10	2.33	2.26	1.85
OPENER-ZS$_{trans}$	1.14	1.23	1.58	2.08	1.82	1.41	1.82	2.23	1.62	1.87	1.63	1.90	1.79	1.70
Supervised (target labels)														
OPENER-Sup	1.10	1.26	0.87	1.76	1.41	1.31	1.75	1.87	1.96	1.97	1.73	1.91	1.78	1.59

Abbreviations follow Table I. Best per column in bold.
§ OWNER is measured at inference only (its one-time 88-min CoNLL-2003 training is amortised, not charged per run).
† CoNLL is its in-domain training source, measured by a separate inference-only run.
‡ Qwen2.5-1.5B-Instruct (4-bit) is on a 150-sentence subsample per set.

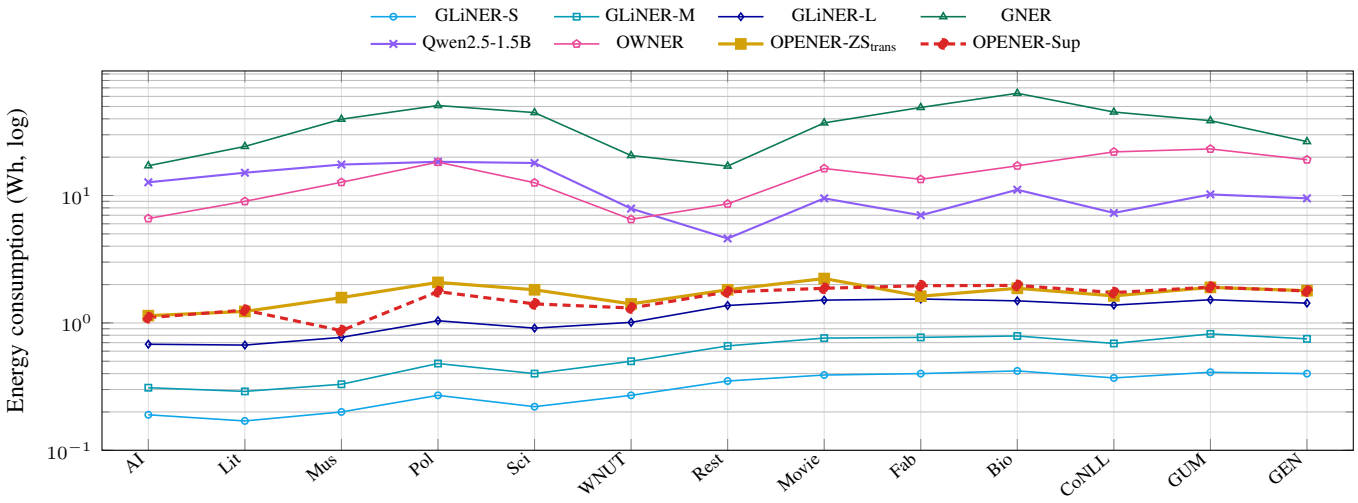

Fig. 4. Per-run energy consumption (Wh, *log* scale) per dataset (Table VII). Same systems and styling as Figure 2. The OPENER variants sit an order of magnitude below OWNER and the generative GNER/Qwen baselines.

TABLE VIII
QUALITY VS EFFICIENCY, 13-SET MEANS.

Model	Params	AMI↑	p50 (ms)↓	Energy (Wh)↓	CO_2 (g)↓
Zero-shot (no target labels)					
GLiNER-S	50M	36.5	21	0.31	0.02
GLiNER-M	200M	37.1	40	0.58	0.03
GLiNER-L	330M	38.9	143	1.18	0.07
GNER	220M	37.9	4226	36.51	2.05
Qwen2.5-1.5B (4-bit)‡	1.5B	26.1	5051	11.4	0.60
OWNER§	294M	34.3	616	14.3	0.74
OPENER-ZS$_{ind}$	467M	$33.8_{\pm 0.7}$	137	1.85	0.10
OPENER-ZS$_{trans}$	467M	$39.5_{\pm 0.1}$	182	1.70	0.10
Supervised (target labels)					
OPENER-Sup	467M	$40.1_{\pm 0.5}$	143	1.59	0.09

AMI ($\times 100$, ↑, OPENER rows: mean $\pm$ std over three retraining seeds). P50 latency (ms), energy (Wh), carbon (gCO_2eq) per run (↓, measured on the released seed-42 run). Parameters (end-to-end OPENER = GLiNER-L detector 330M + embedder 137M = 467M, or 137M on gold, OWNER ≈294M, GLiNER parameter counts from the released checkpoints).
§ OWNER's latency is an amortised wall-clock mean and its one-time 88-min training is not charged per run (Table VI).
‡ Qwen2.5-1.5B-Instruct (4-bit) is on a 150-sentence subsample per set.

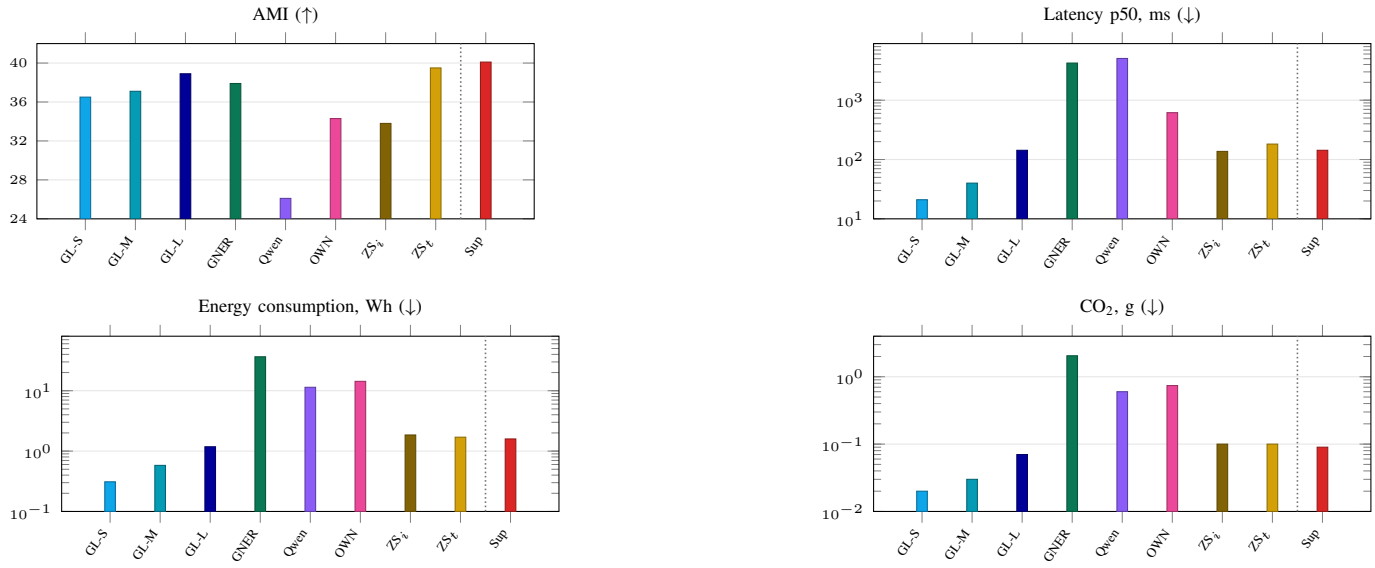

Fig. 5. Per-model means on the four axes (13-set averages, Table VIII). The light dotted line separates the *zero-shot* systems (left) from the *supervised* OPENER-Sup (right). Latency, energy and CO_2 use a log scale. Labels: GL-S/M/L = GLiNER sizes, OWN = OWNER, ZS_i = OPENER-ZS_{ind}, ZS_t = OPENER-ZS_{trans}, Sup = OPENER-Sup.

G. Analysis

a) Where OPENER wins, and why: Two domain patterns stand out across Table V and Figure 2.

First, the detector-fusion gains of OPENER-ZS_{trans}concentrate on the context-rich encyclopaedic CrossNER domains (+14.7 AMI on Music, +10.1 on Science, +8.8 on Politics over the un-fused centroid), where the detector's own zero-shot label is most reliable and lifts OPENER-ZS to detector-level quality (e.g. Music 58.3, edging GLiNER-L's 58.0).

Second, on specialised schemas with opaque, abbreviation-like labels (FabNER, BioNLP), OPENER's contrastive typing maps cryptic surface forms far better than prompt-based naming models, as OWNER: OPENER-Sup leads end-to-end (29.8 and 35.9 AMI), and on gold mentions, the gap widens sharply (57.1/69.2 versus OWNER's 31.9/29.2, Table X).

b) Detection is the bottleneck, worst on specialised domains: Contrasting typing on gold mentions with the end-to-end scores isolates the cost of detection. For OPENER-Sup, the mean drop from gold to end-to-end is 22.2 AMI, but it is far from uniform: it averages 18.8 on the encyclopaedic CrossNER domains against 29.9 on the specialised schemas (BioNLP, MIT-Restaurant, MIT-Movie, FabNER), peaking at 33.3 on BioNLP. The typing head is strong everywhere (gold AMI above 50 on 11 of 13 sets). What caps end-to-end quality is the open-world detector's recall on cryptic, domain-specific surface forms, which confirms detection, not typing, as the dominant bottleneck. The clearest next step for this method is therefore to build and train a stronger mention detector, which we expect to raise the end-to-end scores drastically.

c) What remains hard: The residual typing errors are dominated by confusions between fine-grained, lexically close types within a domain rather than across domains. The most frequent type confusions are DNA vs. protein (BioNLP), the FabNER process labels CONPRI vs. MATE/PRO/MANP, and the Movie attributes TITLE vs. GENRE/RATING: precisely the opaque schemas of Table I where even the surrounding context barely disambiguates the type.

d) Contrastive learning in action: Figure 6 visualises the effect of the two fine-tuning stages on a *held-out* domain (WNUT-17), never seen during contrastive training. Frozen and unmodified, the Nomic embedding intermixes entity types. The contrastive step already pulls same-type mentions together, and the hard-negative mining pass sharpens the boundaries between the types it most often confuses, yielding compact, well-separated clusters. The type geometry learned on CoNLL thus transfers to an unseen domain, which is what makes the cheap label-name and linear typing heads effective.

H. Ablations

We justify each architectural choice with a one-factor-at-a-time search. Starting from the default pipeline, we vary a single component while holding the others fixed and keep the setting with the highest mean AMI. Among embedder variants, an error-driven *hard-negative mining* pass, fine-tuning the contrastive embedder on the entity pairs it most often confuses (e.g. *writer*/*person*, *album*/*band*, *country*/*location*), mixed with correctly-typed mentions for diversity, raises typing-on-gold AMI from 54.0 to 60.1. Scaling the mined triplets (3000 → 8000) and epochs (2 → 3) lifts it further to 62.5 (+8.5 over the contrastive embedder, better on 12 of 13 sets), our selected configuration. Table IX reports this search. The protocol comparison between typing-on-gold and end-to-end is discussed alongside the main results. The remaining axes confirm the rest of the pipeline:

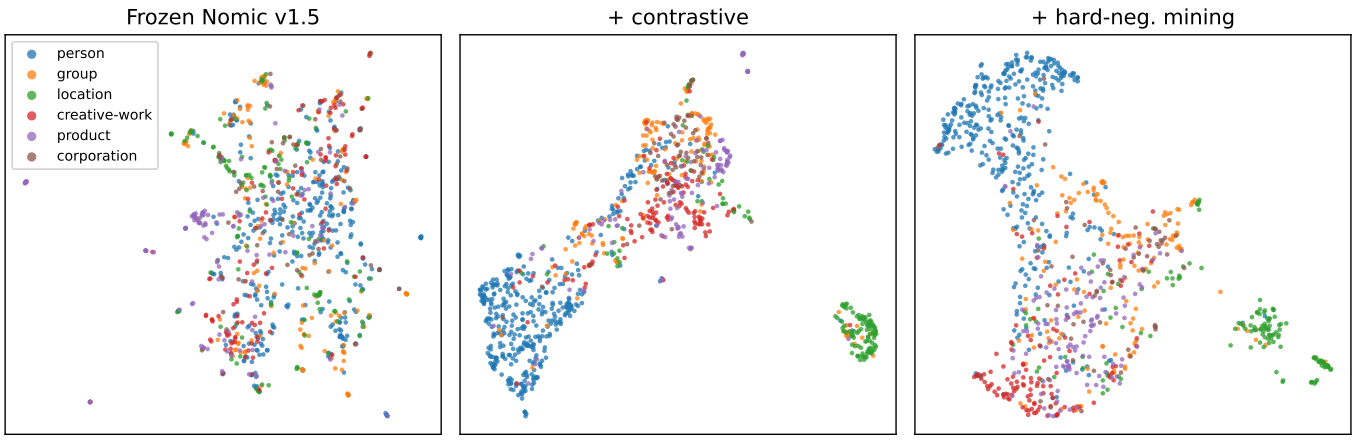

Fig. 6. UMAP of WNUT-17 (held-out) entity embeddings, coloured by gold type, across the three embedder stages: frozen Nomic v1.5, + contrastive fine-tuning, + hard-negative mining. The contrastive stages progressively separate the types into compact clusters on a domain unseen during training.

among frozen ready-to-use base encoders, Nomic v1.5 is the most discriminative for entity typing (25.8 AMI, ahead of e5-base 25.3, mxbai 24.4, MPNet 24.1 and BGE 23.6) and is selected (the heaviest candidate, mxbai-large, is moreover $\sim 8\times$ slower per sentence for no quality gain). Even so, frozen embeddings are only weakly discriminative, so the contrastive step is what makes the space usable ($+28.2$). The detector, which only proposes spans, re-typed by the OPENER head, trades quality for cost (GLiNER-S $\rightarrow$ L: $37.7 \rightarrow 40.2$ AMI at $51 \rightarrow 143$ ms end-to-end), and OPENER's typing on GLiNER-L's spans (40.2) even beats GLiNER-L's own typing (38.9, Table V). A closed-set NER detector (BERT or RoBERTa-NER, CoNLL-trained) used in its place collapses to ~ 0 recall outside PER/ORG/LOC/MISC (MIT, FabNER, BioNLP), dragging macro-F_1 to 17.9/24.6 (vs 30.5) and confirming that open-world detection is required. And for typing a balanced linear SVM edges out a per-class GMM and logistic regression (62.5 vs 56.1 vs 55.6). Replacing the supervised typing head with a label-name nearest-centroid removes target supervision entirely, at a cost of 18.1 AMI ($62.5 \rightarrow 44.4$). This base variant, OPENER-ZS$_{\text{ind}}$, stays on par with the zero-shot OWNER baseline (43.2 vs 43.0 over the three seeds, Table X) while fitting in 293 ms instead of 2.2 s, making it the frugal choice when no target labels are available. A label-free *transductive* refinement of the prototypes, spherical k-means seeded by the label names and run once on the unlabelled test mentions, recovers 7.0 AMI on gold mentions ($44.4 \rightarrow 51.4$, Table IX) and 2.6 end-to-end at the released seed ($34.2 \rightarrow 36.8$, above OWNER's 34.3), widening OPENER-ZS$_{\text{trans}}$'s margin over OWNER while still using no target labels. This is a transductive rather than inductive setting, as it exploits the unlabelled test distribution. We further investigated whether the zero-shot head could be improved by aligning entity mentions and label names in a shared space, trained on the large label vocabulary of Pile-NER [4] ($\approx$4.6k types) with an entity$\leftrightarrow$label-name contrastive objective (Table XI). This produces a strictly domain-agnostic embedder (unlike the selected hard-mining embedder, it never sees any target-side data), yet it does not improve mean AMI (42.2 vs 44.4): the Pile-NER vocabulary lifts macro-F_1 ($+3.8$) and helps encyclopaedic domains (CrossNER-Politics $+14.2$, -Science $+8.6$) but hurts specialised schemas (MIT-Restaurant -15.9, FabNER -11.8, BioNLP -11.8). A second label-name hard-mining pass further degrades it (39.7 AMI): Pile-NER's GPT-generated types include many near-synonyms (e.g. *person*/*per*, *location*/*loc*), so mining "confusions" between them manufactures negatives that pull apart labels that should stay close. Therefore, we retain the target-tuned hard-mining embedder for OPENER-ZS and report the Pile-NER variants as a negative ablation. A separate efficiency axis emerges in the *training* cost (Table IX, Fit): per-sentence inference is governed by the detector alone (truncating the Matryoshka dimension or swapping the classifier leaves it unchanged), whereas fitting the typing head is $7\times$ cheaper at dimension 64 than at 768 (for only a 4.6-point AMI drop, $62.5 \rightarrow 57.9$) and $4.5\times$ cheaper for balanced logistic regression than for the linear SVM, a speed/quality knob orthogonal to inference.

Figure 7 plots typing-on-gold quality against the Matryoshka dimension d. Quality degrades smoothly as d shrinks, with clearly diminishing returns rather than a cliff: d=256 already reaches 61.2 AMI (within 1.3 of the full 62.5) and the curve flattens above it. Truncation therefore trades a little quality for a large cut in the typing-head fit cost, and, crucially, this confirms that the Matryoshka structure *survives* our contrastive fine-tuning: the embedder can still be truncated and renormalised, exposing a graceful cost/quality knob.

To isolate the entity-typing component itself, Table X compares OPENER against OWNER on gold mentions, where detection errors are removed. One cell stands out: OPENER-ZS$_{\text{ind}}$drops to 16.1 AMI on MIT-Movie, with the largest seed variance of the table (± 7.1). We conjecture that the schema's attribute-like label names (e.g. PLOT, REVIEW, TRAILER) carry little lexical signal in a corpus where every mention is movie-related, which leaves the label-name prototypes poorly anchored

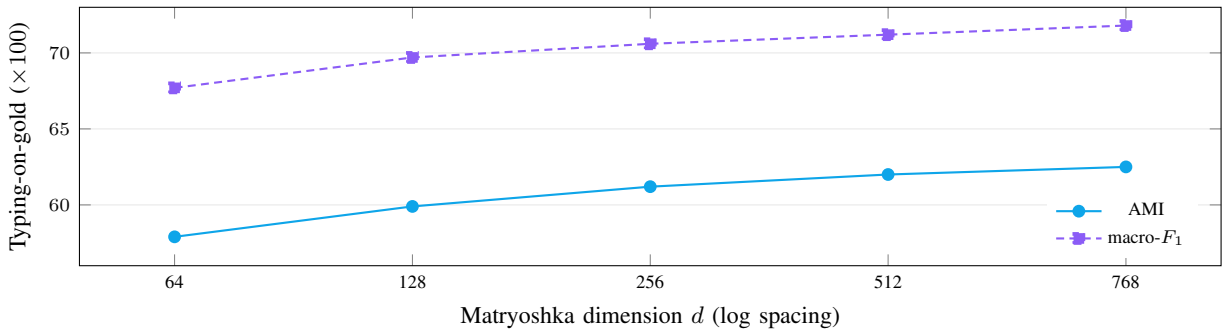

Fig. 7. Typing-on-gold quality vs the Matryoshka truncation dimension d. Both AMI and macro-F_1 degrade smoothly, with diminishing returns rather than a cliff, and flatten above d=256. Truncation is therefore a graceful cost/quality knob, and the Matryoshka structure survives the contrastive fine-tuning. Data from Table IX (dimension axis, released seed, 13-set means, log-spaced d).

and seed-sensitive. The transductive refinement of OPENER-ZS$_{\text{trans}}$, which re-estimates the prototypes from the test mentions themselves, recovers most of this gap (44.8). The higher end-to-end value of OPENER-ZS$_{\text{ind}}$on the same set (32.8, Table V) does not contradict the gold score: the sentinel labels added by the end-to-end protocol reshape both partitions, so the two AMI values are not directly comparable.

TABLE IX

OPENER ARCHITECTURE SEARCH: ONE COMPONENT IS VARIED AT A TIME AROUND THE DEFAULT PIPELINE (**BOLD**).

Configuration	AMI ↑	F_1 ↑	Fit (ms) ↓	p50 (ms) ↓	En. (Wh) ↓	CO_2 (g) ↓
Mention detector (each row: that detector → OPENER embedder + typing head)						
GLiNER-S	37.7	28.8	0.0	51	0.73	0.04
GLiNER-M	38.0	28.7	0.0	70	1.01	0.06
GLiNER-L (open-world)	**40.2**	**30.5**	0.0	143	1.59	0.09
BERT-NER (closed-set) [8]	33.9	17.9	0.0	68	0.58	0.03
RoBERTa-NER (closed-set) [9]	39.4	24.6	0.0	54	0.58	0.04
Entity embedder[*]						
frozen bge-base-en-v1.5 [10]	23.6	41.9	0	43	0.63	0.04
frozen all-mpnet-base-v2 [11]	24.1	42.6	0	52	0.85	0.05
frozen mxbai-embed-large-v1 [12]	24.4	42.5	0	363	3.86	0.25
frozen e5-base-v2 [13]	25.3	43.4	0	39	0.59	0.04
frozen Nomic v1.5 (selected base)	25.8	43.9	0	42	0.63	0.04
+ contrastive (Triplet, CoNLL, 3 ep.)	54.0	63.3	0	42	0.63	0.04
+ hard-negative mining (2 ep., 3k tr.)	60.1	68.8	0	42	0.63	0.04
+ hard-negative mining (3 ep., 8k tr.)	**62.5**	**71.8**	0	42	0.63	0.04
Matryoshka dimension[§]						
64	57.9	67.7	299	42	0.63	0.04
128	59.9	69.7	562	42	0.63	0.04
256	61.2	70.6	926	42	0.63	0.04
512	62.0	71.2	1556	42	0.63	0.04
768	**62.5**	**71.8**	2176	42	0.63	0.04
Entity typing[‡]						
GMM (per-class)	56.1	65.3	527	42	0.63	0.04
LogReg	55.6	53.5	546	42	0.63	0.04
LogReg (balanced)	57.8	67.5	485	42	0.63	0.04
LinearSVC	61.7	66.5	2041	42	0.63	0.04
LinearSVC (balanced)	**62.5**	**71.8**	2217	42	0.63	0.04
Nearest centroid (zero-shot)[¶]	44.4	46.8	293	42	0.63	0.04
+ transductive refine[¶]	51.4	48.5	293	42	0.63	0.04

Metrics. AMI and macro-F_1 ($\times 100$, ↑). Fit (ms) is the typing-head training time. Latency, energy and carbon are per row (↓). All rows use the released seed (42).

Protocol, which differs per axis. On the *mention-detector* axis each row is OPENER's *end-to-end* AMI with that detector, which only proposes spans that the OPENER head re-types (unlike the standalone systems of Table V). The [*] embedder, [‡] typing and [§] dimension axes are instead *typing-on-gold* over the 13 sets.

Cost. Latency and energy are the deployed end-to-end cost on the mention-detector axis, and the constant embedder-plus-typing cost on the other axes.

[§] Truncates the Matryoshka vector after encoding.

[¶] Zero-shot nearest-centroid head (no target labels). Its Fit is the one-off prototype-embedding cost.

TABLE X

ENTITY TYPING ON GOLD MENTIONS (DETECTION BYPASSED): OPENER VS OWNER (AMI $\times 100$, ↑).

Model	AI	Lit	Mus	Pol	Sci	WNUT	Rest	Movie	Fab	Bio	CoNLL	GUM	GEN	Avg
Zero-shot (no target labels)														
OWNER	55.7	54.3	59.8	63.2	63.1	30.5	29.3	31.8	31.9	29.2	52.5[†]	30.0	27.2	43.0
OPENER-ZS_{ind}	$38.7_{\pm 3.6}$	$57.7_{\pm 3.3}$	$54.7_{\pm 1.7}$	$57.4_{\pm 5.8}$	$57.8_{\pm 1.6}$	$31.3_{\pm 2.6}$	$38.1_{\pm 4.6}$	$16.1_{\pm 7.1}$	$30.4_{\pm 5.1}$	$51.1_{\pm 2.9}$	$63.9^{\dagger}_{\pm 3.0}$	$33.7_{\pm 3.8}$	$31.0_{\pm 2.5}$	$43.2_{\pm 1.3}$
OPENER-ZS_{trans}	$46.1_{\pm 2.0}$	$63.5_{\pm 2.3}$	$67.0_{\pm 1.9}$	$66.4_{\pm 1.2}$	$59.6_{\pm 2.0}$	$32.7_{\pm 2.8}$	$47.3_{\pm 4.8}$	$44.8_{\pm 1.1}$	$33.9_{\pm 3.3}$	$49.8_{\pm 0.3}$	$70.7^{\dagger}_{\pm 1.7}$	$41.5_{\pm 2.5}$	$38.8_{\pm 1.6}$	$50.9_{\pm 0.4}$
Supervised (target labels)														
OPENER-Sup	$\mathbf{57.6}_{\pm 0.9}$	$\mathbf{66.0}_{\pm 2.0}$	$\mathbf{72.3}_{\pm 1.5}$	$\mathbf{73.6}_{\pm 1.8}$	$\mathbf{69.4}_{\pm 2.2}$	$\mathbf{38.8}_{\pm 1.0}$	$\mathbf{63.2}_{\pm 2.0}$	$\mathbf{67.7}_{\pm 1.9}$	$\mathbf{57.1}_{\pm 2.0}$	$\mathbf{69.2}_{\pm 1.8}$	$73.3^{\dagger}_{\pm 1.6}$	$\mathbf{49.8}_{\pm 0.6}$	$\mathbf{51.5}_{\pm 0.2}$	$\mathbf{62.3}_{\pm 1.0}$

Reading. Abbreviations follow Table I. Bold marks the best value per column. Systems are grouped by supervision.

Statistics. OPENER rows report mean ± std over the three embedder-retraining seeds (subsection IV-D). OWNER is a single training run.

Typing heads. OPENER-ZS_{ind} is the un-refined nearest centroid and OPENER-ZS_{trans} adds the transductive refinement. Note that the detector fusion of Table V is end-to-end-only, so it is inactive here, where detection is bypassed. OPENER-Sup is the supervised upper bound.

Outlier. The low OPENER-ZS_{ind} MIT-Movie cell (16.1 ± 7.1) is analysed in subsection IV-H: the schema's attribute-like label names give the prototypes little lexical signal, and the sentinel-based end-to-end score of Table V is not directly comparable to this gold score.

[†] CoNLL-2003 is in-domain for OWNER (training source) and OPENER (contrastive source) and is not bolded.

TABLE XI
ZERO-SHOT TYPING: EFFECT OF THE CONTRASTIVE EMBEDDER ON OPENER-ZS$_{\text{IND}}$(TYPING-ON-GOLD, 13-SET MEANS AT THE RELEASED SEED (42)).

Embedder	Training data	AMI	F_1	Acc
hard-neg. mining (selected)	CoNLL + targets' train	**44.4**	46.8	53.6
Pile-NER (entity↔label)	Pile-NER only	42.2	**50.6**	**53.9**
+ label-name hard mining	Pile-NER only	39.7	41.4	44.7

The nearest label-name centroid head is fixed, only the embedder changes. AMI, macro-F_1, accuracy ($\times 100$, ↑). Best per column in bold. The selected hard-negative-mining embedder (target-tuned) is compared with Pile-NER [4] variants that are strictly domain-agnostic (never see target data). We report Pile-NER as a negative ablation (subsection IV-H).

V. CONCLUSION

We presented OPENER, an open-world NER pipeline assembled entirely from ready-to-use components, a frozen detector, a pretrained Matryoshka embedder sharpened by a light contrastive fine-tuning with error-driven hard-negative mining, and a swappable typing head, with no encoder trained from scratch. The modular design is part of the result rather than an implementation detail. Because the three stages are independently swappable, we could search the architecture one axis at a time (Table IX) and select every component on measured evidence instead of by assumption, and the same modularity keeps the embedder's Matryoshka structure usable as a compute/quality knob even after our contrastive fine-tuning (Figure 7).

Both operating points of the typing head perform strongly, at opposite ends of the supervision axis. Our zero-shot head OPENER-ZS, label-name prototypes refined transductively and fused with the detector's own predictions, needs no target labels at typing time and is still the best of the compared zero-shot systems end-to-end (39.5 AMI, all AMI scores being reported $\times 100$ as in the tables), ahead of GLiNER-L, GNER, Qwen2.5 and OWNER, at one to two orders of magnitude lower energy than the trained-encoder and generative baselines. When a domain's entity classes are fixed and labelled, a balanced linear probe, OPENER-Sup, goes further. On gold mentions, the contrastive space is highly discriminative, and OPENER-Sup is the most accurate system in our benchmark, leading on all 13 sets and surpassing a zero-shot transfer of OWNER (62.3 vs 43.0 AMI). It also attains the best end-to-end AMI of the compared systems (40.1 AMI*100).

Both results rest on a benchmark that is itself a contribution: 13 open-world NER datasets on which quality, per-sentence latency and energy are reported jointly, typing-on-gold is kept separate from end-to-end, and the supervision asymmetry between systems is stated rather than hidden. Every OPENER number is averaged over three full embedder-retraining seeds.

Two findings carry beyond the specific system. First, an open-world typing head need not train its own encoder: a general-purpose embedding, once sharpened by a light contrastive step, is already discriminative enough to type entities, and the cost that does matter is fitting the cheap typing head, not the embedder. Second, the three-axis evaluation locates the open-world bottleneck in *detection* rather than typing: typing on gold mentions reaches 62.3 AMI but end-to-end scores drop to 40.1 as detector recall falls on specialised domains. So, improving the detector, not the typing head, is the clearest axis for future gains. The evaluation also exposes efficiency gaps that an accuracy-only comparison would hide.

The approach has clear limits. End-to-end quality is capped by detector recall on specialised domains, the shared bottleneck, not by the typing head. Supervised typing needs target-domain labels, and while OPENER-ZS removes them at typing time it depends on the quality of the label names and, in its transductive form, on access to the unlabelled test mentions. The inductive zero-shot head is correspondingly weaker. Finally, our experiments run on deliberately modest hardware: a single consumer GPU with 6 GB of VRAM, subject to thermal throttling. This is the very setting that makes OPENER deployable, but it also bounded our experimental budget: despite an already broad benchmark (13 datasets, three axes and extensive ablations), we cap each test split at 1000 sentences. The quality scores themselves are nonetheless well characterised: retraining the full embedder under three seeds leaves the benchmark means nearly unchanged (std of 0.1 to 1.3 AMI on the thirteen-set means, subsection IV-D), inference is deterministic at a fixed seed, and a test-set bootstrap confirms per-dataset stability (bootstrap std of at most 1.5 AMI, Table III). Only latency and energy fluctuate ($\pm 15\%$) and are therefore read as averages rather than precise points.

Future work includes improving mention detection on specialised domains to lift the end-to-end ceiling, an inductive zero-shot head that does not rely on the test distribution, and extending the decoupled detect-then-type design beyond NER to entity linking and relation extraction.

ACKNOWLEDGMENTS

In accordance with the symposium policy on the use of generative AI, we disclose that a general-purpose large-language-model coding assistant was used during this work. Its use was limited to engineering and editorial support: scaffolding and debugging experiment scripts, drafting and copy-editing the manuscript, and assisting with exploratory data analysis. The research questions, the experimental design, the choice of baselines and datasets, the execution of all experiments, and the

interpretation of the results are the authors' own. All reported numbers were produced by the released code and verified by the authors, who take full responsibility for the scientific content and the final wording of the paper.

REFERENCES

[1] N. T. H. Nguyen, M. Miwa, and S. Ananiadou, "Span-based named entity recognition by generating and compressing information," in *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2023, pp. 1984–1996.

[2] U. Zaratiana, N. Tomeh, P. Holat, and T. Charnois, "GLiNER: Generalist model for named entity recognition using bidirectional transformer," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Volume 1: Long Papers*. Association for Computational Linguistics, 2024, pp. 5364–5376.

[3] Y. Ding, J. Li, P. Wang, Z. Tang, B. Yan, and M. Zhang, "Rethinking negative instances for generative named entity recognition," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 3461–3475.

[4] W. Zhou, S. Zhang, Y. Gu, M. Chen, and H. Poon, "UniversalNER: Targeted distillation from large language models for open named entity recognition," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[5] O. Sainz, I. García-Ferrero, R. Agerri, O. Lopez de Lacalle, G. Rigau, and E. Agirre, "GoLLIE: Annotation guidelines improve zero-shot information-extraction," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[6] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2025.

[7] P.-Y. Genest, P.-E. Portier, E. Egyed-Zsigmond, and M. Lovisetto, "OWNER: Toward unsupervised open-world named entity recognition," *IEEE Access*, vol. 13, pp. 50 077–50 105, 2025.

[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019, pp. 4171–4186.

[9] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[10] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general chinese embeddings," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2024, pp. 641–649.

[11] K. Song, X. Tan, T. Qin, J. Lu, and T.-Y. Liu, "MPNet: Masked and permuted pre-training for language understanding," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[12] X. Li and J. Li, "AoE: Angle-optimized embeddings for semantic textual similarity," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 1825–1839.

[13] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, "Text embeddings by weakly-supervised contrastive pre-training," *arXiv preprint arXiv:2212.03533*, 2022.

[14] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, "Nomic embed: Training a reproducible long context text embedder," *Transactions on Machine Learning Research (TMLR)*, 2025, published in TMLR 02/2025. OpenReview: https://openreview.net/forum?id=IPmzyQSiQE.

[15] A. Kusupati, G. Bhatt, A. Rege, M. Wallingford, A. Sinha, V. Ramanujan, W. Howard-Snyder, K. Chen, S. Kakade, P. Jain, and A. Farhadi, "Matryoshka representation learning," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.

[16] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer, "Neural architectures for named entity recognition," in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2016, pp. 260–270.

[17] X. Ma and E. Hovy, "End-to-end sequence labeling via bi-directional LSTM-CNNs-CRF," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016, pp. 1064–1074.

[18] J. Yu, B. Bohnet, and M. Poesio, "Named entity recognition as dependency parsing," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 6470–6476.

[19] X. Li, J. Feng, Y. Meng, Q. Han, F. Wu, and J. Li, "A unified MRC framework for named entity recognition," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 5849–5859.

[20] E. F. Tjong Kim Sang and F. De Meulder, "Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition," in *Proceedings of the Seventh Conference on Natural Language Learning (CoNLL)*, 2003, pp. 142–147.

[21] J. Li, A. Sun, J. Han, and C. Li, "A survey on deep learning for named entity recognition," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 1, pp. 50–70, 2022.

[22] P. He, J. Gao, and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing," in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[23] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, pp. 1–67, 2020.

[24] R. Aly, A. Vlachos, and R. McDonald, "Leveraging type descriptions for zero-shot named entity recognition and classification," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP)*, 2021, pp. 1516–1528.

[25] N. Ding, G. Xu, Y. Chen, X. Wang, X. Han, P. Xie, H.-T. Zheng, and Z. Liu, "Few-NERD: A few-shot named entity recognition dataset," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP)*, 2021, pp. 3198–3213.

[26] X. Wei, X. Cui, N. Cheng, X. Wang, X. Zhang, S. Huang, P. Xie, J. Xu, Y. Chen, M. Zhang, Y. Jiang, and W. Han, "ChatIE: Zero-shot information extraction via chatting with ChatGPT," *arXiv preprint arXiv:2302.10205*, 2023.

[27] R. Cai, J. Lu, Z. Chen, B. Xu, and Z. Hao, "Handling missing entities in zero-shot named entity recognition: Integrated recall and retrieval augmentation," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL)*, 2025, pp. 10 790–10 802.

[28] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[29] Y. Tang, R. Hasan, and T. Runkler, "FsPONER: Few-shot prompt optimization for named entity recognition in domain-specific scenarios," in *Proceedings of the 27th European Conference on Artificial Intelligence (ECAI)*. IOS Press, 2024.

[30] S. Zhang, B. Cao, and J. Fan, "KCL: Few-shot named entity recognition with knowledge graph and contrastive learning," in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*. ELRA Language Resource Association, 2024, pp. 9681–9692.

[31] S. S. S. Das, A. Katiyar, R. J. Passonneau, and R. Zhang, "CONTaiNER: Few-shot named entity recognition via contrastive learning," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022, pp. 6338–6353.

[32] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.

[33] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823.

[34] P. Lai, Z. Zhang, W. Zhang, F. Fu, and B. Cui, "Enhancing unsupervised sentence embeddings via knowledge-driven data augmentation and Gaussian-decayed contrastive learning," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*. Association for Computational Linguistics, 2025, pp. 4919–4940.

[35] G. Wang, C. Li, W. Wang, Y. Zhang, D. Shen, X. Zhang, R. Henao, and L. Carin, "Joint embedding of words and labels for text classification," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018, pp. 2321–2331.

[36] Y. Zhang, W. Chen, and L. Ma, "Contrastive learning with Gaussian embeddings and self-attention for few-shot named entity recognition," *Applied Sciences*, vol. 15, no. 21, p. 11819, 2025.

[37] X. Hu, Y. Jiang, A. Liu, Z. Huang, P. Xie, F. Huang, L. Wen, and P. S. Yu, "Entity-to-text based data augmentation for various named entity recognition tasks," in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023, pp. 9072–9087.

[38] Y. Abbahaddou, F. D. Malliaros, J. F. Lutzeyer, A. M. Aboussalah, and M. Vazirgiannis, "Graph neural network generalization with Gaussian mixture model based augmentation," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 102–125.

[39] D. A. Reynolds, "Gaussian mixture models," in *Encyclopedia of Biometrics*, S. Z. Li and A. K. Jain, Eds. Springer US, 2009, pp. 659–663.

[40] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 39, no. 1, pp. 1–22, 1977.

[41] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 3645–3650.

[42] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.

[43] D. Patterson, J. Gonzalez, U. Hölzle, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "The carbon footprint of machine learning training will plateau, then shrink," *Computer*, vol. 55, no. 7, pp. 18–28, 2022.

[44] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, and J. Pineau, "Towards the systematic reporting of the energy and carbon footprints of machine learning," *Journal of Machine Learning Research*, vol. 21, pp. 1–44, 2020.

[45] C.-J. Wu, R. Raghavendra, U. Gupta, B. Acun, N. Ardalani, K. Maeng, G. Chang, F. Aga, J. Huang, C. Bai *et al.*, "Sustainable AI: Environmental implications, challenges and opportunities," in *Proceedings of Machine Learning and Systems (MLSys)*, 2022.

[46] A. S. Luccioni, Y. Jernite, and E. Strubell, "Power hungry processing: Watts driving the cost of AI deployment?" in *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2024, pp. 85–99.

[47] S. Luccioni, V. Schmidt, A. Lacoste, and T. Dandres, "Quantifying the carbon emissions of machine learning," in *NeurIPS 2019 Workshop on Tackling Climate Change with Machine Learning*, 2019. [Online]. Available: https://www.climatechange.ai/papers/neurips2019/22

[48] N. X. Vinh, J. Epps, and J. Bailey, "Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance," *Journal of Machine Learning Research*, vol. 11, pp. 2837–2854, 2010.

[49] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[50] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin, "LIBLINEAR: A library for large linear classification," *Journal of Machine Learning Research*, vol. 9, pp. 1871–1874, 2008.

[51] Z. Liu, Y. Xu, T. Yu, W. Dai, Z. Ji, S. Cahyawijaya, A. Madotto, and P. Fung, "CrossNER: Evaluating cross-domain named entity recognition," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 15, 2021, pp. 13 452–13 460.

[52] L. Derczynski, E. Nichols, M. van Erp, and N. Limsopatham, "Results of the WNUT2017 shared task on novel and emerging entity recognition," in *Proceedings of the 3rd Workshop on Noisy User-generated Text (W-NUT)*, 2017, pp. 140–147.

[53] J. Liu, P. Pasupat, S. Cyphers, and J. Glass, "Asgard: A portable architecture for multilingual dialogue systems," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2013, pp. 8386–8390.

[54] A. Kumar and B. Starly, "FabNER: Information extraction from manufacturing process science domain literature using named entity recognition," *Journal of Intelligent Manufacturing*, vol. 33, no. 8, pp. 2393–2407, 2022.

[55] J.-D. Kim, T. Ohta, Y. Tsuruoka, Y. Tateisi, and N. Collier, "Introduction to the bio-entity recognition task at JNLPBA," in *Proceedings of the International Joint Workshop on Natural Language Processing in Biomedicine and its Applications (NLPBA/BioNLP)*, 2004, pp. 70–75.

[56] A. Zeldes, "The GUM corpus: Creating multilayer resources in the classroom," *Language Resources and Evaluation*, vol. 51, no. 3, pp. 581–612, 2017.

[57] T. Aoyama, S. Behzad, L. Gessler, L. Levine, J. Lin, Y. J. Liu, S. Peng, Y. Zhu, and A. Zeldes, "GENTLE: A genre-diverse multilayer challenge set for english NLP and linguistic evaluation," in *Proceedings of the 17th Linguistic Annotation Workshop (LAW-XVII)*, 2023, pp. 166–178.

[58] J.-D. Kim, T. Ohta, Y. Tateisi, and J. Tsujii, "GENIA corpus—a semantically annotated corpus for bio-textmining," *Bioinformatics*, vol. 19, no. Suppl. 1, pp. i180–i182, 2003.

[59] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*. Curran Associates, Inc., 2023.

[60] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[61] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2020, pp. 38–45.

[62] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.